
Apprentissage par renforcement pour la navigation visuelle

Tuteur entreprise : Youssef ALJ
Tuteur école : Said MOUSSAOUI

Najwa AIDAT - Le 15 septembre 2026
Avril - Août 2026

Plan

- | | |
|---|--|
| 1 | Contexte et problématique |
| 2 | RL et 3DGS |
| 3 | Reconstruction de la scène avec 3DGS |
| 4 | Reproduction du pipeline de navigation visuelle (AI2-THOR) |
| 5 | Construction du graphe 3DGS et entraînement sur room |
| 6 | Approche de navigation continue |
| 7 | Conclusion et perspectives |

Contexte et problématique

Question : Peut-on remplacer un simulateur synthétique coûteux par une reconstruction 3DGS d'une scène réelle ?

Problème 1 : Sim to real gap

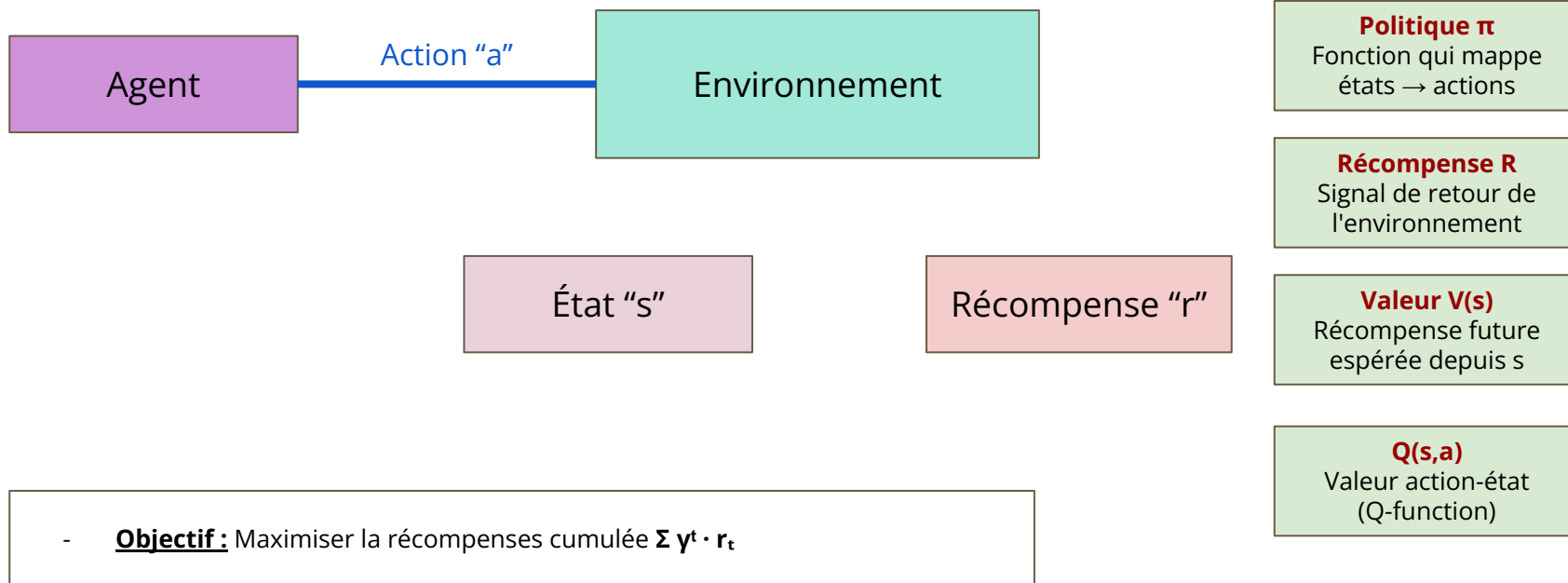
- Agents entraînés en simulation peinent dans les scènes réelles
- Simulateurs classiques (AI2-THOR) : synthétiques écart visuel
- Pipeline coûteux à mettre en place

Problème 2 : Asservissement visuel

- Fonctionne bien mais calcul de la matrice d'interaction coûteux
- Bassin d'attraction limité (pose initiale proche requise)
- Sensible aux changements d'illumination

Reinforcement Learning

- Principe



Les problèmes connus du RL

- Ce dont on parle rarement, mais qui bloque en pratique :

Problème	Explication	Cas pratique
Manque de généralisation vers de nouvelles cibles.	Le modèle apprend très bien une tâche précise... mais échoue dès qu'on change d'environnement ou de cible.	Un agent entraîné à naviguer dans une maison A ne sait plus naviguer dans une maison B, même similaire.
Inefficacité des données : apprentissage lent et coûteux.	Le RL nécessite énormément d'interactions pour apprendre : millions de pas avant de converger.	Chaque interaction coûte du temps, de l'énergie, ou risque d'endommager du matériel. Sample inefficiency = coût prohibitif.

3D Gaussian Splatting

Photo 2D
N poses & poses caméra

SFM
Nuage de point sparse

Gaussienne 3D
 μ, Σ, α , couleur (SH)

Mise à jour des
Gaussiennes

Rendu 2D
Projection Splatting

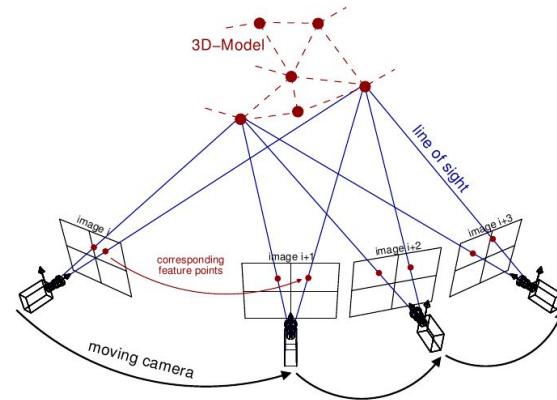
Comparaison
Image synth. vs vraie

Correction
Backprop → ajuste μ, Σ ,
 α, c

Scène 3D finale
Rendu temps réel ~100 FPS

Boucle d'optimisation

Convergence



Source : Logessiva, Medium – "Understanding and Exploring 3D Gaussian Splatting" (2024)

Reconstruction de la scène avec 3DGS

Dataset:

- Le dataset Mip-NeRF 360 comprend plusieurs scènes capturées à 360° en environnements intérieurs et extérieurs, telle que :
 - bicycle
 - bonsai
 - counter
 - garden
 - kitchen
 - **room**
 - Stump
 - ...

Chaque scène est accompagnée de :

- 311 images capturées disponibles sous plusieurs facteurs de sous-échantillonnage : images, images_2, images_4 et images_8.
- Les poses de caméra reconstruites par COLMAP (fichiers cameras.bin, images.bin, points3D.bin)
- Un nuage de points 3D épars issu de la reconstruction SfM

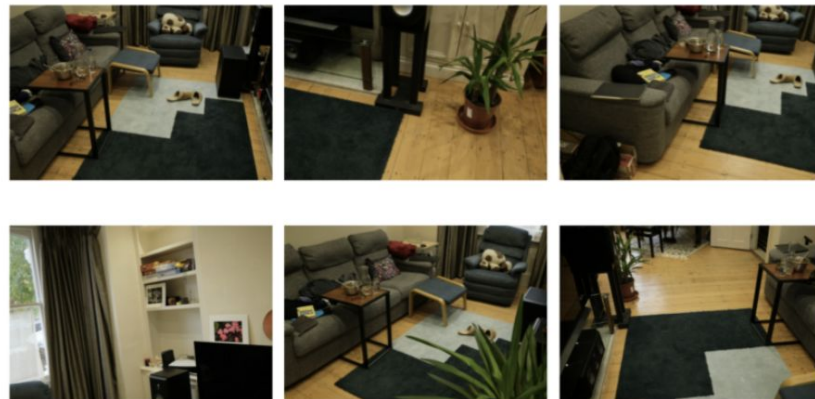


Figure 1: Exemple d'images du Dataset

Reconstruction de la scène Avec 3DGS

Résultats visuels:

Pipeline suivi :

- 311 images (images_4/) + poses COLMAP
- Conversion COLMAP → transforms.json
- Entraînement Splatfacto : 30 000 itérations sur Toubkal / Kaggle
- Export → fichier .ply
- Visualisation via Supersplat¹

1. <https://playcanvas.com/supersplat/editor/>

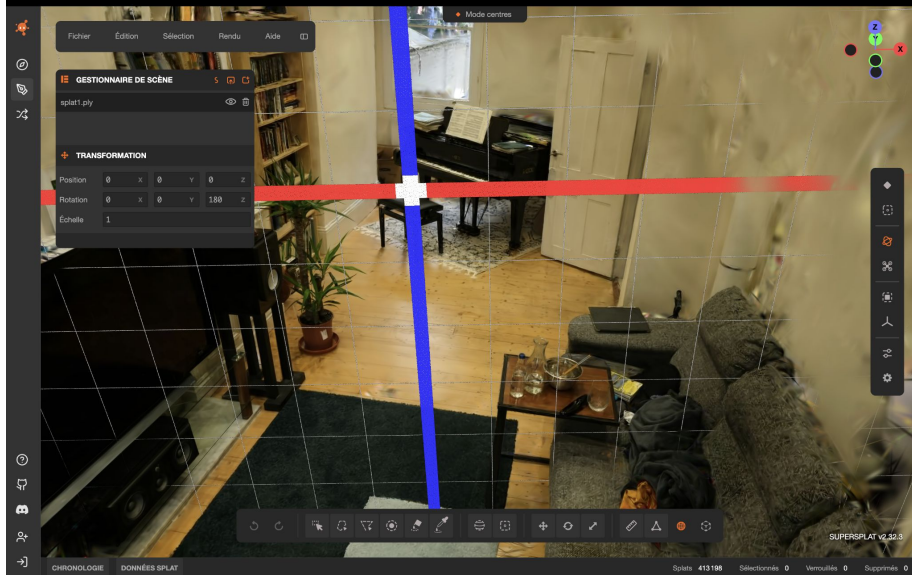


Figure 2: Visualisation de la scène 3D construite

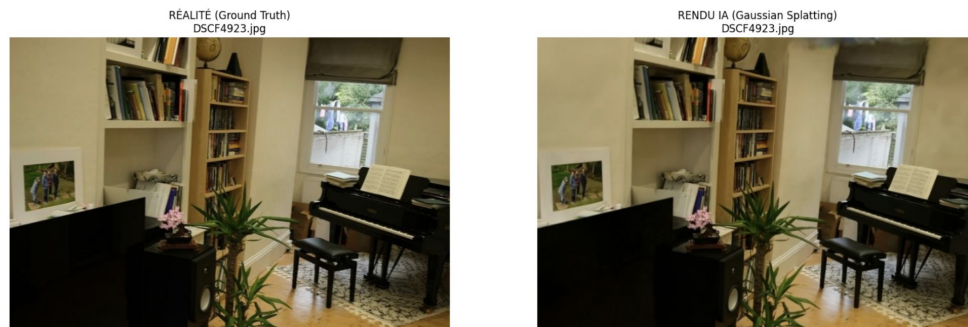


Figure 3: Comparaison entre les images réelles et les rendues s

Résultats visuels:



Figure 4: Comparaison entre les images réelles, les rendus 3DGS et les cartes de différence pixel à pixel

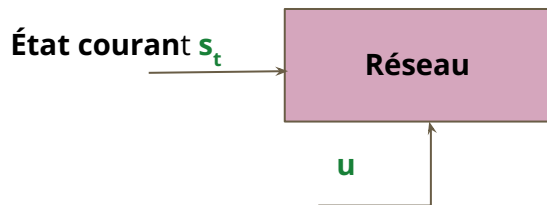
- PSNR = 31.49 dB
- Le modèle reconstruit les surfaces de manière lisse, agissant ainsi comme un filtre de restauration.

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right)$$

Reproduction du pipeline de navigation visuelle (AI2-THOR)

Approche Standard:

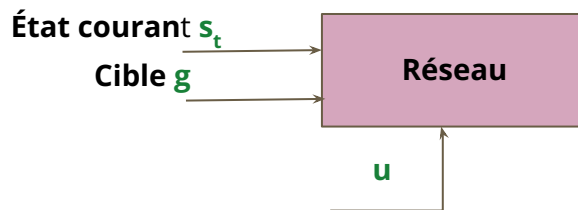
- $\pi(s_t | u)$
- L'objectif est "codé en dur" dans les poids u .
- Un réseau = Une seule cible. Nécessite un ré-entraînement complet pour chaque nouvel objet



Cible g encodée dans les paramètres

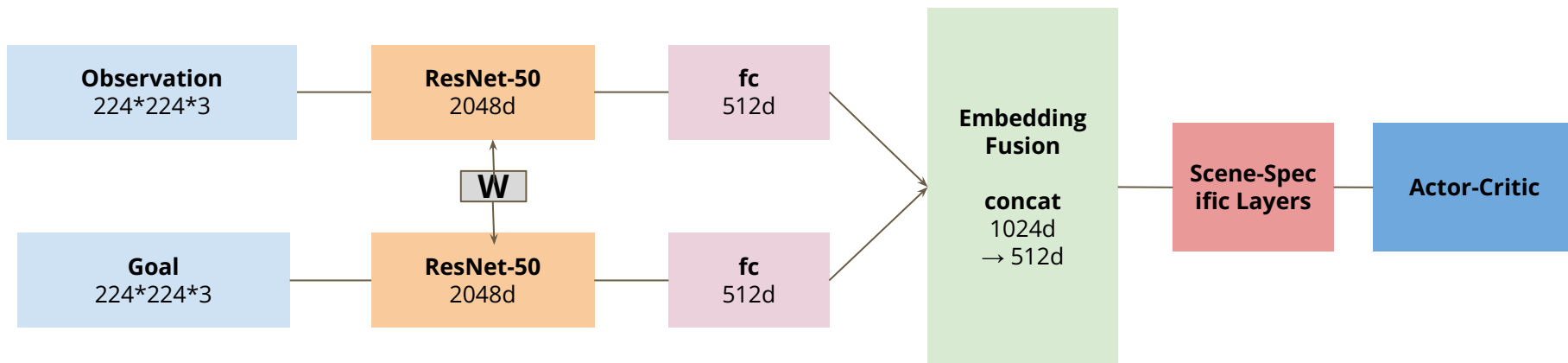
Approche de l'article:

- $\pi(s_t, g | u)$
- La cible g devient une **entrée explicite** du réseau.



Reproduction du pipeline de navigation visuelle (AI2-THOR)

Architecture du réseau:



- $a = \pi(s_t, g \mid u)$
- Le ResNet-50 a déjà été pré-entraîné sur ImageNet => ResNet est gelé

Reproduction du pipeline de navigation visuelle (AI2-THOR)

Détection de la cible et entraînement:

- La navigation est discrète
- L'agent dispose de 4 actions : MoveForward, RotateRight, RotateLeft et MoveBackward
- L'épisode est considéré comme réussi uniquement lorsque l'agent atteint exactement la position de la cible (terminal_state_id)
- La fonction de récompense est définie comme suit :

$$r_t = \begin{cases} +10 & \text{si la nouvelle position} = \text{position cible} \\ -0.1 & \text{en cas de collision} \\ -0.01 & \text{à chaque pas normal} \end{cases}$$



Figure 5: Exemple de graphe de navigation AI2-THOR original (data/bathroom_02.h5) — 45 nœuds, yaw = 0°.

Reproduction du pipeline de navigation visuelle (AI2-THOR)

Résultats et comparaison:

Scène	Longueur moyenne (pas)
bathroom_02	7.8
bedroom_04	15.1
living_room_08	16.7
kitchen_02	21.6
Moyenne (notre ré-implémentation)	15.3
Moyenne article	210.7

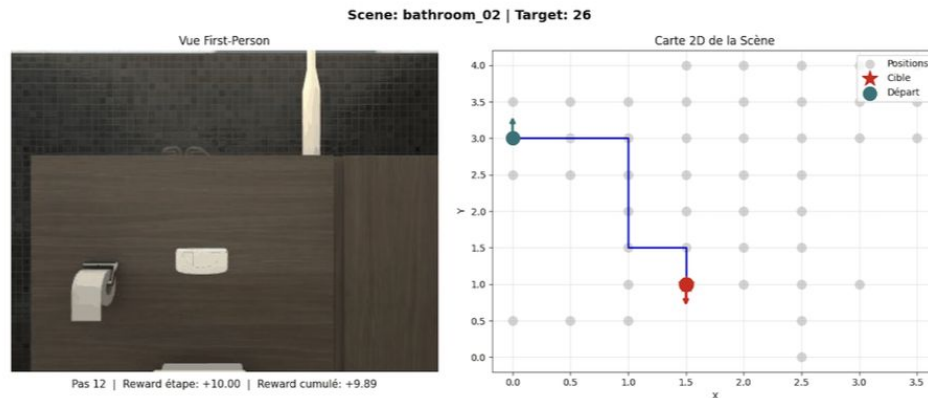


Figure 6: Exemple de navigation de l'agent A3C sur la scène bathroom_02 (cible 26)

- La borne théorique du plus court chemin **17.6 pas**

Construction du graphe 3DGS et entraînement sur room

Extraction des poses COLMAP:

- Les 311 poses de caméra sont lues depuis les fichiers binaires COLMAP
- Une grille uniforme de pas $\Delta = 1$ m est générée à l'intérieur de l'enveloppe convexe

Rendu des vues synthétiques

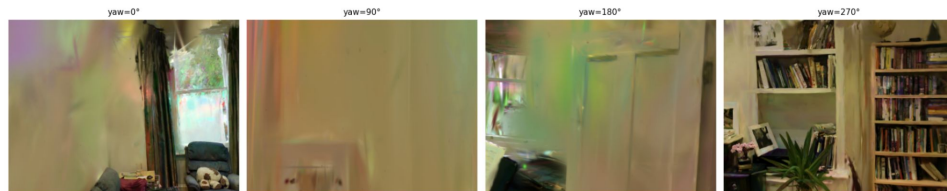


Figure 9: Vues synthétisées par le renderer 3DGS pour les quatre orientations cardinales (yaw = 0°, 90°, 180°, 270°)

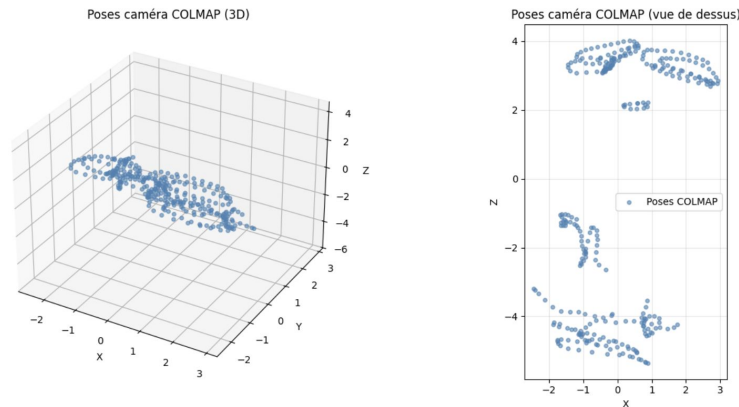


Figure 7: Disposition des poses caméra dans la scène room

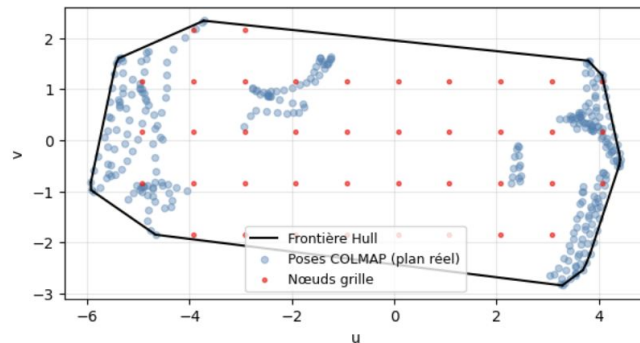


Figure 8: Enveloppe convexe des poses COLMAP et grille de navigation

Construction du graphe 3DGS et entraînement sur room

Génération du fichier HDF5:

- Le graphe de navigation, les features ResNet-50, les distances du plus court chemin (shortest path distance) et les métadonnées sont sauvegardés dans un fichier **scene_3dgs.h5** compatible avec le framework ICRA 2017.
- Pour l'entraînement:
 - Cibles : 5 cibles fixes (24, 60, 120, 108, 48)
 - 500 000 itérations
 - Fonction de récompense : +10 à la cible, -0.1 en collision, -0.01 par pas

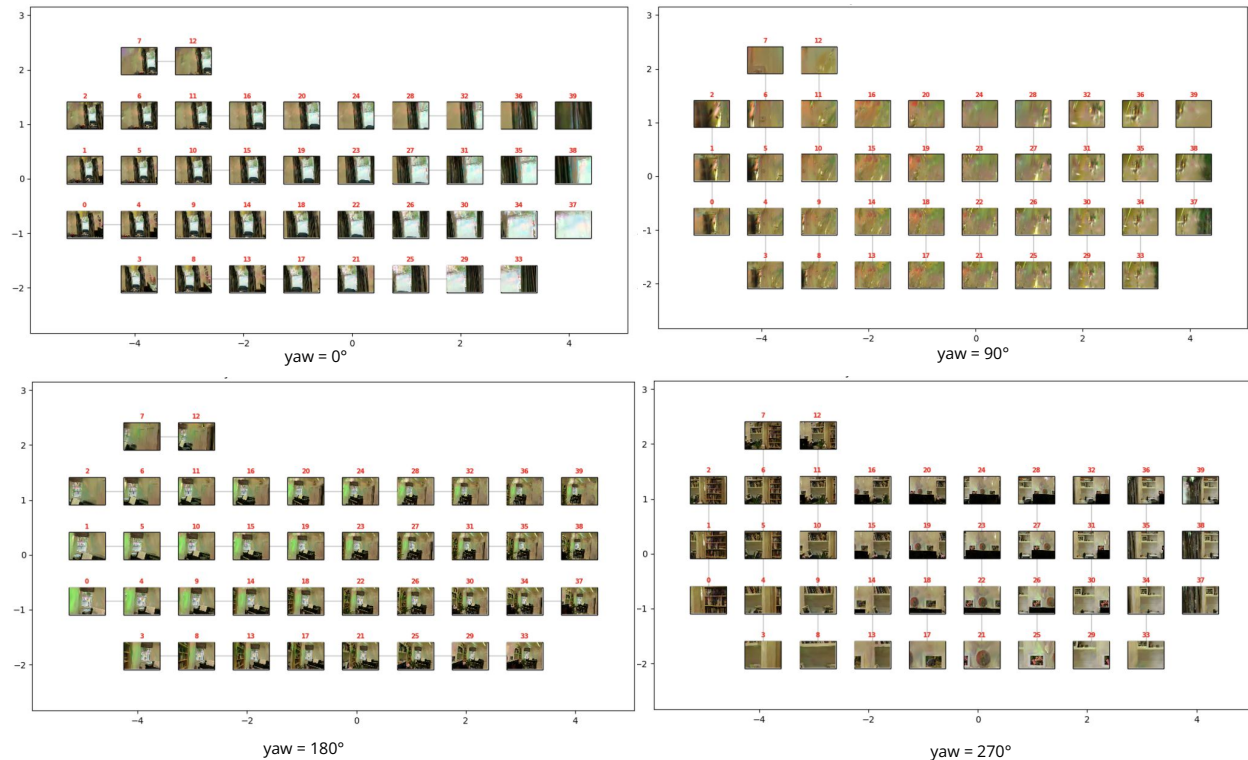
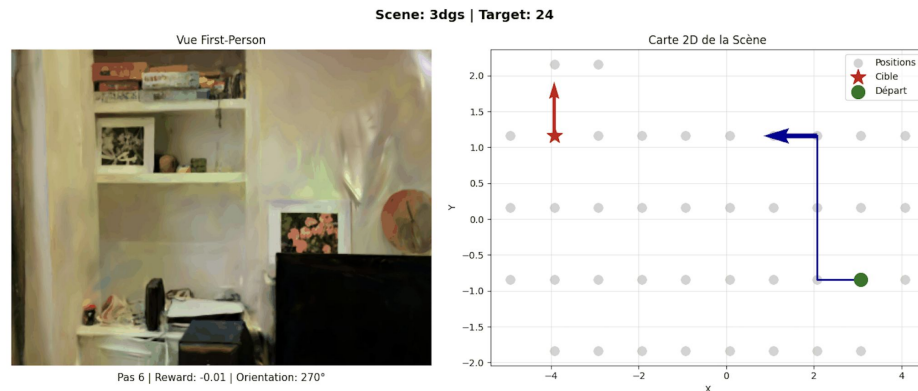


Figure 10 Graphes de navigation par le render 3DGS pour les quatre orientations cardinales (yaw = 0°, 90°, 180°, 270°).

Construction du graphe 3DGS et entraînement sur room

Résultats :

Cible	Reward moyen	Longueur moyenne (pas)	Collisions
#24	9.90	10.44	0.01
#48	9.92	8.78	0.06
#60	9.91	9.17	0.07
#108	9.93	7.21	0.06
#120	9.92	7.86	0.11
Moyenne	9.92	8.69	0.06



Approche de navigation continue

Phase 1 : A3C vs PPO (1 rotation)

L'agent reçoit comme entrée un état numérique regroupant sa position courante, la position cible (X,Y,Z), et l'angle de lacet courant(YAW) et cible (normalisés).

On remarque que:

- Le PPO converge plus vite et de façon nettement plus fiable que A3C.
- A3C est instable

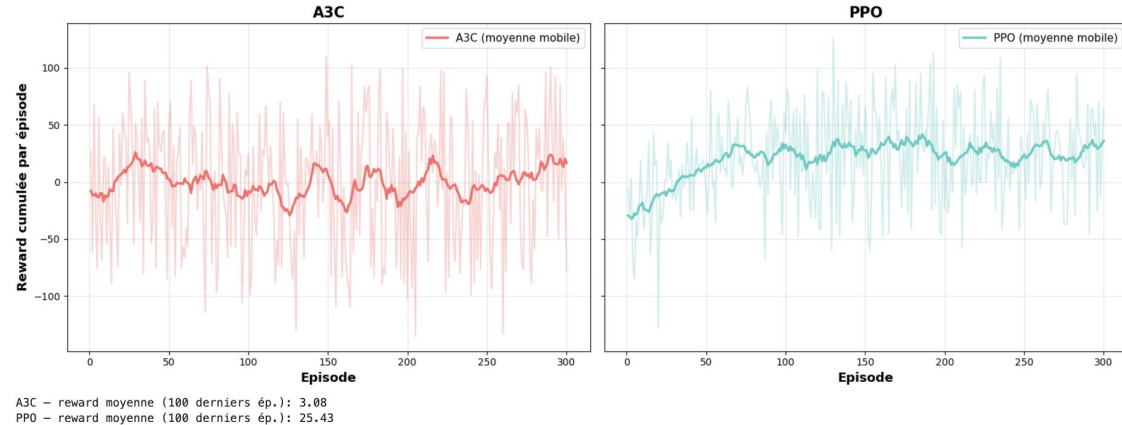


Figure 12 Reward par épisode

Approche de navigation continue

Seconde phase : comparaison A3C / PPO avec orientation complète:

La récompense repose sur un écart combiné entre l'état courant et l'état cible, qui associe un terme de position et un terme d'orientation.

$$\tilde{d}_{\text{pos}} = \frac{\|\mathbf{p}_t - \mathbf{p}^*\|_2}{\text{diam}(\mathcal{P})} \quad d_{\text{orient}} = \arccos\left(\frac{\text{tr}(\mathbf{R}_t^T \mathbf{R}^*) - 1}{2}\right)$$

$$\Delta(\alpha, \beta) = |((\alpha - \beta + 180^\circ) \bmod 360^\circ) - 180^\circ|$$

$$\tilde{d}_{\text{orient}} = \frac{1}{3} \left(\frac{\Delta(\psi_t, \psi^*)}{180^\circ} + \frac{\Delta(\theta_t, \theta^*)}{180^\circ} + \frac{\Delta(\phi_t, \phi^*)}{180^\circ} \right)$$

$$D_t = \tilde{d}_{\text{pos}} + \tilde{d}_{\text{orient}}$$

On remarque que:

- Les deux algorithmes sont devenus beaucoup moins stables qu'avec une seule orientation => problème dans la façon d'intégrer ces trois rotations dans le signal d'apprentissage.

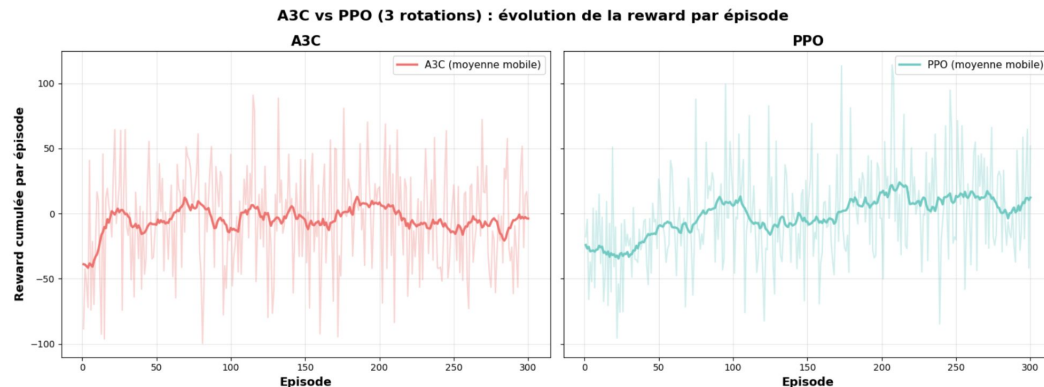


Figure 13 Reward par épisode

Approche de navigation continue

Nouvelle représentation de l'orientation : quaternions

Représentation par quaternions + erreur relative

PPO — 3000 épisodes :

- Reward : 0 → 35-38 (quasi monotone, sans effondrement)
- Distance : 3 m → < 0.5 m (700 premiers épisodes)
- Seuil succès (0.08 m) atteint à l'épisode ~1300
- Haute précision (0.02 m) stabilisée après épisode 2000

A3C — 3000 épisodes :

- Reward : -5 → +4 (variance élevée, pas de stabilisation)
- Distance : 3.1 m → 1.9 m seulement
- Seuils 0.08 m et 0.02 m jamais atteints

→ PPO converge, A3C échoue — même représentation, même tâche

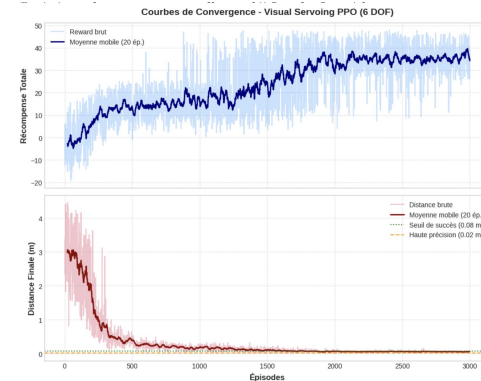
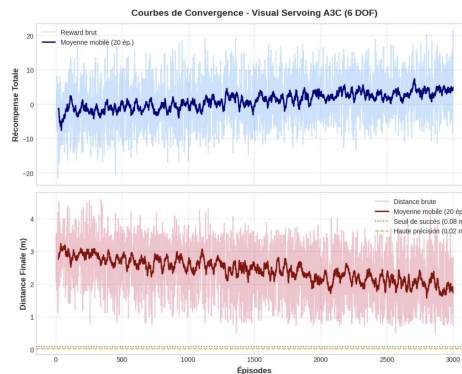


Figure 14 Reward et erreur par épisode

Approche de navigation continue

Troisième phase : observation par image (espace des caractéristiques)

- Remplacer la pose (position et orientation) explicite par les caractéristiques visuelles des images
- ResNet-18 pré-entraîné ImageNet, poids gelés

$$\mathbf{z} = \frac{f_{\text{ResNet}}(I)}{\|f_{\text{ResNet}}(I)\|_2} \in \mathbb{R}^{512}$$

- Distance :

$$d_{\text{feat}} = (1 - \mathbf{z}_t \cdot \mathbf{z}^*) \times 100$$

- Récompense :

$$r_t = k \cdot (d_{\text{feat},t-1} - d_{\text{feat},t}) + r_{\text{term}} \cdot \mathbb{I}[d_{\text{feat},t} < \varepsilon]$$

On remarque que pour le:

- PPO reward stable ~80-110 dès épisode ~500
- A3C distance : 13 m → 5-6 m (seuils jamais atteints)

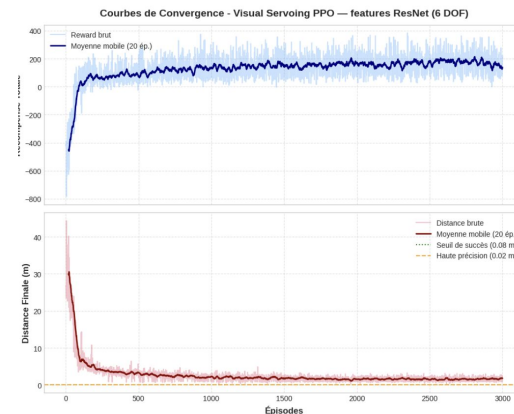
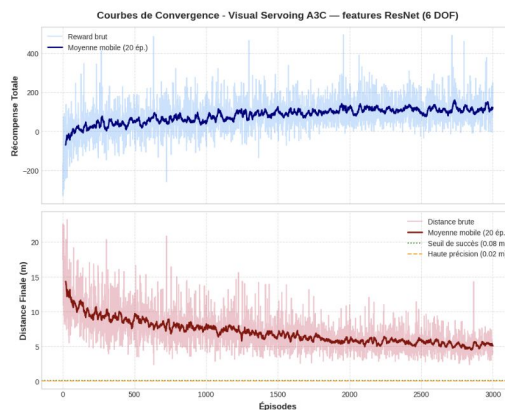


Figure 15 Reward et erreur par épisode

Merci pour votre attention

Questions ?

Références

- Yuke Zhu, Roozbeh Mottaghi, Eric Kolve, Joseph J. Lim, Abhinav Gupta, Li Fei-Fei, and Ali Farhadi. Target-driven Visual Navigation in Indoor Scenes using Deep Reinforcement Learning, September 2016. URL : <http://arxiv.org/abs/1609.05143>.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-nerf 360 : Unbounded anti-aliased neural radiance fields. CVPR, 2022.
- Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Justin Kerr, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Appel, David McAllister, and Angjoo Kanazawa. Nerfstudio : A modular framework for neural radiance field development. ACM SIGGRAPH, 2023
- Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Tim Harley, Timothy P Lillicrap, David Silver, and Koray Kavukcuoglu. Asynchronous Methods for Deep Reinforcement Learning. In Proceedings of The 33rd International Conference on Machine Learning, 2016.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Transactions on Graphics, 42(4) :1–14, August 2023. ISSN 0730-0301, 15577368. URL <https://dl.acm.org/doi/10.1145/3592433>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms, August 2017. URL <http://arxiv.org/abs/1707.06347> arXiv :1707.06347 [cs.LG].